PCS Lineup — Documentação Técnica & Arquitetura

> **Versão:** 2.0.0 · **Última atualização:** 2026-10-18 · **Gerado a partir da análise rigorosa do código-fonte.**

Sumário

1. Visão Geral Técnica
2. Stack Tecnológica & Dependências
3. Estrutura de Diretórios
4. Variáveis de Ambiente & Configurações
5. Diagrama de Arquitetura do Sistema
6. Diagrama de Sequência — Fluxo Operacional Crítico
7. Modelo de Dados (PostgreSQL)
8. Motor Algorítmico Preditivo & Inteligência Náutica
9. Guia de Inicialização & Ambiente Local
10. Deploy em Produção (Docker & Dokploy)
11. Pontos de Atenção & Inconsistências Identificadas

---

1. Visão Geral Técnica

O **PCS Lineup** é um sistema web operacional de missão crítica, alta densidade informacional e análise preditiva desenvolvido para o **Centro de Controle Operacional (CCO)** da **Companhia Docas de São Sebastião (CDSS)**.

A aplicação gerencia a programação de atracação de navios (*line-up* comercial e de cabotagem), orquestrando em tempo real:

- A ocupação do **Cais Comercial Único (Berço 101)** e dos **Berços Internos de Apoio (201 a 204)**.
- O cálculo determinístico da duração de operação baseado em pranchas mínimas regulamentares e históricas por tipo de carga.
- A dedução preditiva de paralisações por precipitação para granéis sensíveis (chuva horária).
- A análise hidrodinâmica de **Folga Abaixo da Quilha (UKC - Under Keel Clearance)** considerando maré astronômica (DHN), telemetria ao vivo da maré e duplo calado náutico (Entrada vs. Saída).
- A sincronização automatizada com o **SISPORT Oficial** da CDSS e a reconciliação inteligente com a **Pauta de Manobras da Praticagem de São Paulo (SP Pilots)**.
- Telemetria de posicionamento satelital via transponder **AIS (DataDocked)** e geofencing de berço.

A arquitetura é dividida em dois blocos principais:

- **Backend:** API REST monolítica construída em Node.js (TypeScript) com Express e driver assíncrono `postgres`.
- **Frontend:** SPA (Single Page Application) moderna em React 19 + TypeScript + Vite + Tailwind CSS v4, com suporte a temas Dark/Light, gráfico Gantt SVG/HTML customizado, cartografia Leaflet/CARTO e painéis de telemetria.

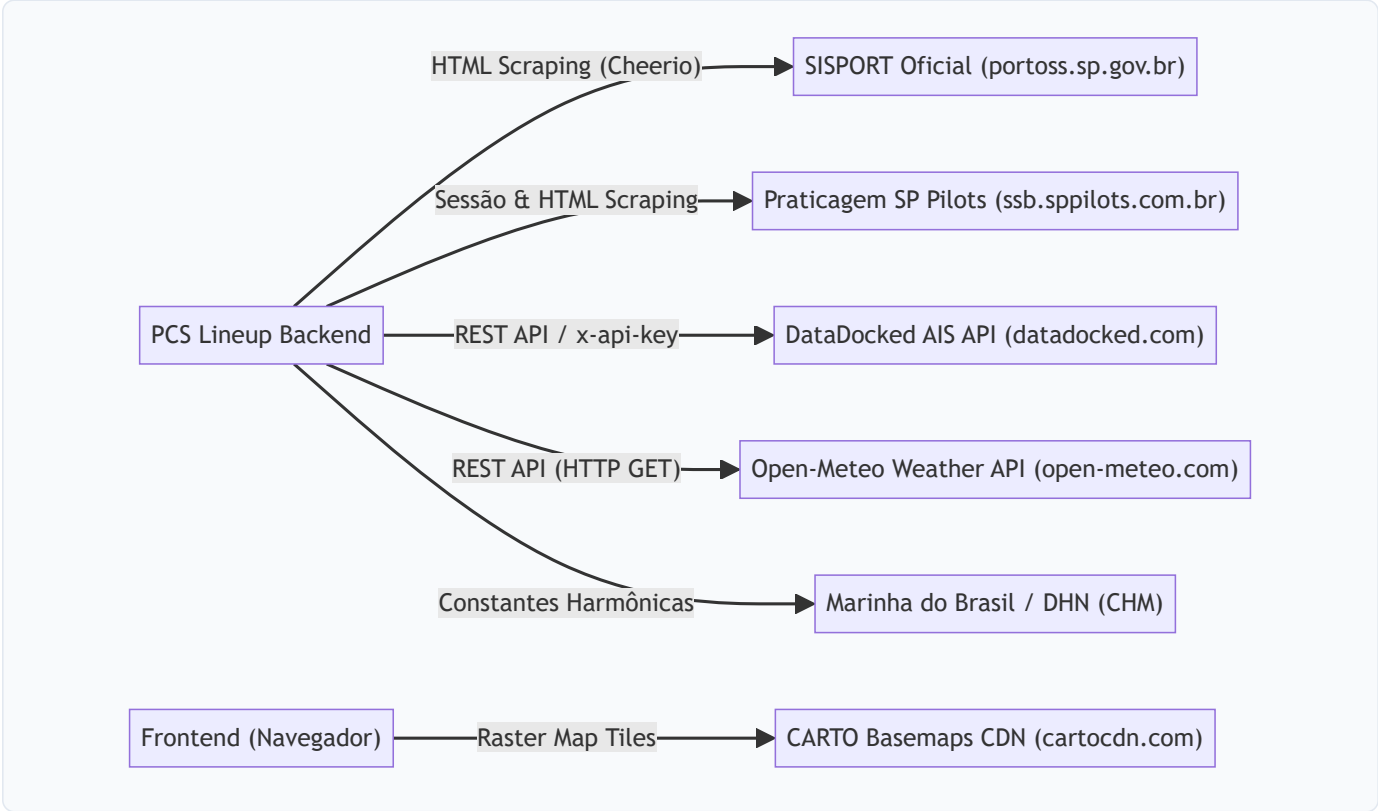
---

2. Stack Tecnológica & Dependências

2.1. Tabela de Tecnologias

Camada	Tecnologia	Versão	Propósito / Papel Arquitetural
Runtime	Node.js	`>=20.0.0` (LTS 22 recomendada)	Ambiente de execução JavaScript/TypeScript do backend e build de assets.
Linguagem (Backend)	TypeScript	`^5.7.3`	Tipagem estática rigorosa (`strict mode`), interfaces compartilhadas de domínio marítimo.
Framework HTTP	Express	`^4.21.2`	Roteamento REST, middlewares de CORS, JSON parsing e graceful shutdown.
Driver de Banco	`postgres` (porsager)	`^3.4.9`	Driver nativo TypeScript para PostgreSQL com tagged template literals e connection pooling.
Banco de Dados	PostgreSQL	`16-alpine` / `18`	Banco relacional com suporte a tipos nativos, índices parciais e JSONB.
Web Scraping & HTML	Cheerio	`^1.2.0`	Parsing e extração resiliente de dados de páginas legadas do SISPORT e Praticagem.
Testes Automatizados	Vitest	`^5.0.5`	Suíte de testes unitários determinísticos (39 testes cobrindo engine, math, geofence e parsers).
Frontend Framework	React	`^19.2.8`	Biblioteca de componentes declarativos para interface de alta densidade do CCO.
Build Tool (Frontend)	Vite	`^8.3.0`	Bundler ESM ultrarrápido para desenvolvimento e empacotamento de produção.
Estilização	Tailwind CSS (v4)	`^4.3.3`	Utility-first CSS com `@tailwindcss/postcss` , tokens corporativos e suporte a temas Dark/Light.
Cartografia & GIS	Leaflet	`^1.9.4`	Renderização do mapa do Canal de São Sebastião, boias, berços e poligonais da ANTAQ.
Tiles de Mapa	CARTO Basemaps	API v1	Mapas base náuticos de alta precisão (Positron, Dark Matter, Voyager, Satellite).
Ícones	Lucide React	`^1.47.0`	Ícones SVG consistentes para o cockpit do CCO.
Linter Frontend	Oxlint	`^1.81.0`	Linter estático de alta velocidade em Rust para JavaScript/TypeScript.
Contêineres	Docker & Compose	Multi-stage (Node 22)	Padronização de build (`builder` e `runner`) e orquestração de microsserviços.

2.2. Serviços Externos e Provedores de Dados



1. **SISPORT Oficial (`portoss.sp.gov.br`):**
   - Fonte primária oficial de escalas comerciais da CDSS.
   - Extrai: IMO, nome do navio, tipo, mercadoria, volume, calado, armazém, previsão de chegada, início de atracação, fim de atracação e % de progresso da operação no berço.
2. **Praticagem de São Paulo — SP Pilots (`ssb.sppilots.com.br`):**
   - Coleta a **Pauta de Manobras e Serviços** (manobras de Entrada `E` , Saída `S` e Mudança `M` , categorias: Encerradas, Em Andamento, Confirmadas, Confirmadas Agente e Solicitadas).
   - Coleta a **Telemetria de Maré ao Vivo** (nível da água em metros, tendência e próximos extremos).
3. **DataDocked AIS Telemetria Satelital (`datadocked.com`):**
   - Coleta a posição satelital em tempo real (latitude, longitude, SOG, COG, status de navegação, sinal de antena) e o ETA transmitido pelo transponder da embarcação.
4. **Open-Meteo Forecast API (`api.open-meteo.com`):**
   - Fornece previsão horária de precipitação (`precipitation` em mm/h) para as coordenadas do Porto de São Sebastião (`lat=-23.8045, lon=-45.3970`), alimentando o cálculo de paralisação por chuva.
5. **Diretoria de Hidrografia e Navegação — DHN / CHM:**
   - Tábua astronômica oficial de extremos de preamar e baixa-mar do Canal de São Sebastião, persistida na tabela `tabua_mares` .
6. **CARTO Basemaps (`carto.com`):**
   - Provedor de tiles vetoriais/raster em alta definição para renderização cartográfica via Leaflet.

---

3. Estrutura de Diretórios

Abaixo está o mapeamento dos componentes de código-fonte da aplicação:

```
d:\DEV\CDSS-Lineup\
├── .env.example                  # Modelo de variáveis de ambiente do projeto
├── Dockerfile                    # Multi-stage build Docker (builder + runner enxuto)
├── docker-compose.yml            # Orquestração local de PostgreSQL 16 + App Express
├── package.json                  # Manifest do backend, scripts npm e dependências Node
├── tsconfig.json                 # Configurações do compilador TypeScript para backend
│
├── src/                          # CÓDIGO-FONTE DO BACKEND (Node.js + Express)
│   ├── server.ts                 # Ponto de entrada, middlewares, inicialização e graceful shutdown
│   ├── version.ts                # Definição e exportação da versão ativa (APP_VERSION)
│   ├── database/
│   │   ├── connection.ts         # Pool de conexões PostgreSQL com driver 'postgres'
│   │   └── init.ts               # DDL de criação de tabelas, índices e migrações defensivas
│   ├── routes/
│   │   └── api.ts                # Mapeamento centralizado de todas as rotas REST (/api/*)
│   ├── controllers/
│   │   ├── lineupController.ts   # Handlers para lineup, navios, clima, marés e histórico
│   │   ├── praticagemController.ts # Handlers para pauta e sincronização da Praticagem
│   │   └── sisportConfigController.ts # Handlers para agendador/worker do SISPORT
│   ├── services/
│   │   ├── engine.ts             # Motor matemático: duração, cascata, PMD, folgas e acurácia
│   │   ├── bercoGeoService.ts    # Geofencing espacial: identificação de berço por coordenadas
│   │   ├── tideService.ts        # Modelagem náutica: UKC, duplo calado e interpolação harmônica
│   │   ├── praticagemService.ts  # Scraper e parser da Pauta de Manobras da SP Pilots
│   │   ├── spPilotsService.ts    # Scraper e parser da telemetria de maré ao vivo da SP Pilots
│   │   ├── sisportSync.ts        # Scraper e normalizador de escalas do SISPORT Oficial
│   │   ├── sisportSchedulerService.ts # Worker periódico em background com Mutex lock
│   │   ├── syncAllService.ts     # Orquestrador de sincronização completa (SISPORT + Praticagem + AIS)
│   │   ├── navioMatcherService.ts # Resolução de IMO via casamento difuso de nomes
│   │   ├── datadocked.ts         # Integração com API satelital AIS DataDocked + mock local
│   │   └── weather.ts            # Integração com Open-Meteo e cálculo de horas de chuva
│   ├── scripts/
│   │   ├── importNavios.ts       # Script de carga inicial do catálogo naval (Navios.csv)
│   │   └── syncSisport.ts        # Script CLI avulso para sincronização do SISPORT
│   └── types/
│       └── index.ts              # Tipos TypeScript, interfaces de domínio e modelos de resposta
│
├── client/                       # CÓDIGO-FONTE DO FRONTEND (React 19 + Vite)
│   ├── package.json              # Dependências do cliente web
│   ├── vite.config.ts            # Configurações do Vite, proxy reverso e build
│   ├── index.html                # Ponto de entrada HTML do SPA
│   └── src/
│       ├── main.tsx              # Bootstrap do React 19
│       ├── App.tsx               # Orquestrador de estado global, abas e polling periódico
│       ├── index.css             # Estilos globais Tailwind CSS v4 e temas Dark/Light
│       ├── context/
│       │   └── ThemeContext.tsx   # Provedor de contexto para alternância Dark/Light
│       ├── components/
│       │   ├── Header.tsx        # Barra superior com KPIs de ocupação, clima e controles de sincronização
│       │   ├── GanttChart.tsx    # Gráfico Gantt CCO com fitas de berços, modos Essencial/Detalhado e curva DHN
│       │   ├── GanttShipDetailsModal.tsx # Prontuário náutico flutuante completo da embarcação
│       │   ├── NavioModal.tsx    # Modal de criação e edição completa de escalas de navios
│       │   ├── CanalMap.tsx      # Mapa Leaflet náutico interativo do Canal e Porto
│       │   ├── AdminQueue.tsx    # Painel de gestão de fila, prioridades e reordenação
│       │   ├── PraticagemView.tsx# Painel dedicado à Pauta de Manobras e Tráfego do Porto
│       │   ├── ConfirmDeleteModal.tsx # Modal de confirmação segura para exclusão de escalas
│       │   └── TideWaveTrack.tsx # Componente de renderização da esteira de maré
│       ├── services/
│       │   └── api.ts            # Camada de comunicação HTTP do cliente com o backend
│       └── types/
│           └── index.ts          # Interfaces e contratos do frontend
│
├── docs/                         # DOCUMENTAÇÃO DO PROJETO & ASSETS DE DADOS
│   ├── ARQUITETURA.md            # Este documento (Documentação Técnica & Arquitetura)
│   ├── API.md                    # Mapeamento completo de contratos e endpoints HTTP da API
│   ├── DESIGN.md                 # Design System, paleta de cores e tipografia corporativa CCO
│   ├── Navios.csv                # Catálogo naval oficial com mais de 16.200 navios e IMOs
│   └── Área do Porto Organizado de São Sebastião.kmz # Poligonal oficial ANTAQ
│
└── tests/
    └── engine.test.ts            # Suíte de 39 testes com Vitest (regras matemáticas e determinísticas)
```

4. Variáveis de Ambiente & Configurações

O sistema lê suas configurações a partir de arquivos `.env` na raiz do projeto (gerenciado por `dotenv`).

4.1. Tabela de Variáveis

Variável	Obrigatória	Padrão / Fallback	Propósito / Descrição	Exemplo Válido
`PORT`	Não	`3000`	Porta TCP em que o servidor Express escuta conexões HTTP.	`3000`
`HOST`	Não	`0.0.0.0`	Interface de rede para vinculação do servidor HTTP.	`0.0.0.0`
`NODE_ENV`	Não	`development`	Ambiente de execução (`development` , `production` , `test`).	`production`
`CLIENT_PORT`	Não	`5173`	Porta local utilizada pelo Vite em desenvolvimento.	`5173`
`DATABASE_URL`	**Sim** (Opção A)	*Nenhum*	URL completa de conexão ao PostgreSQL (preferida em produção e Dokploy).	`postgresql://usr:pwd@host:5432/cdss`
`POSTGRES_USER`	**Sim** (Opção B)	`cdsslineup`	Usuário do banco caso `DATABASE_URL` não seja informada.	`cdsslineup`
`POSTGRES_PASSWORD`	**Sim** (Opção B)	`cdsslineup_secret`	Senha do banco caso `DATABASE_URL` não seja informada.	`sua_senha_segura`
`POSTGRES_DB`	**Sim** (Opção B)	`cdsslineup`	Nome da base de dados PostgreSQL.	`cdsslineup`
`POSTGRES_PORT`	Não	`5432`	Porta TCP do serviço PostgreSQL.	`5432`
`POSTGRES_HOST`	Não	`postgres` / `localhost`	Hostname do contêiner ou servidor PostgreSQL.	`postgres` ou `localhost`
`SP_PILOTS_USER`	Não	`docasssb`	Usuário para autenticação no portal da Praticagem de SP.	`docasssb`
`SP_PILOTS_PASS`	Não	*Confidencial*	Senha para autenticação no portal da Praticagem de SP.	`sua_senha_praticagem`

Variável	Obrigatória	Padrão / Fallback	Propósito / Descrição	Exemplo Válido
`SP_PILOTS_URL`	Não	`https://ssb.sppilots.com.br`	URL base do portal da Praticagem de São Sebastião.	`https://ssb.sppilots.com.br`
`DATADOCKED_API_KEY`	Não	*Nenhum (usa mock)*	Chave da API DataDocked para telemetria satelital AIS sob demanda.	`dd_live_abcdef123456`
`VITE_CARTO_API_KEY`	Não	*Nenhum (tiles std)*	Chave do CARTO para tiles de mapa em alta definição.	`carto_apikey_xyz789`
`SISPORT_AUTO_SYNC_ENABLED`	Não	`true`	Habilita o worker periódico em background do SISPORT (`true` / `false`).	`true`
`SISPORT_SYNC_INTERVAL_MINUTES`	Não	`15`	Intervalo em minutos entre coletas automáticas do SISPORT (mín: 2, máx: 1440).	`15`
`SISPORT_SYNC_MODE`	Não	`SISPORT_ONLY`	Modo padrão do worker: `SISPORT_ONLY` ou `FULL_SYNC` (porto + AIS).	`SISPORT_ONLY`

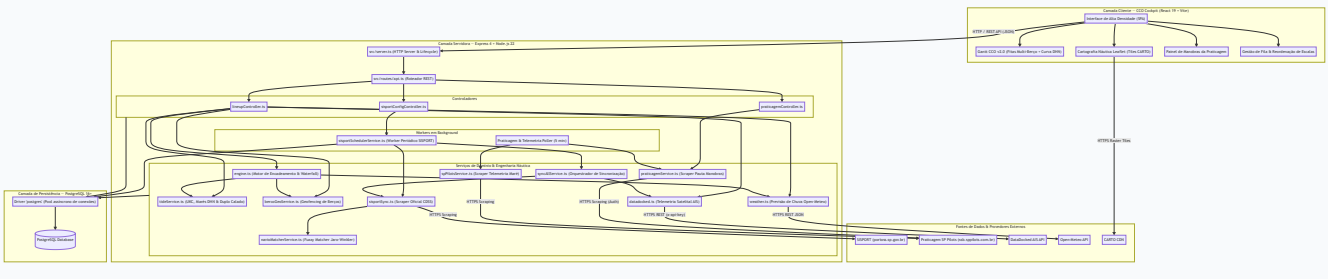
4.2. Variáveis de Compatibilidade e Avisos

> ⚠️ **Inconsistências identificadas no código vs. `.env.example` :**
>
> 1. `src/database/connection.ts` aceita chaves alternativas para o banco: `DB_USER` , `DB_PASSWORD` , `DB_HOST` , `DB_PORT` e `DB_NAME` . Recomenda-se padronizar no uso de `DATABASE_URL` ou prefixo `POSTGRES_*` .
> 2. `client/vite.config.ts` aceita `CARTO_API_KEY` como fallback para `VITE_CARTO_API_KEY` .
> 3. `client/vite.config.ts` injeta a versão da aplicação via variável `VITE_APP_VERSION` (lida de `package.json`).

---

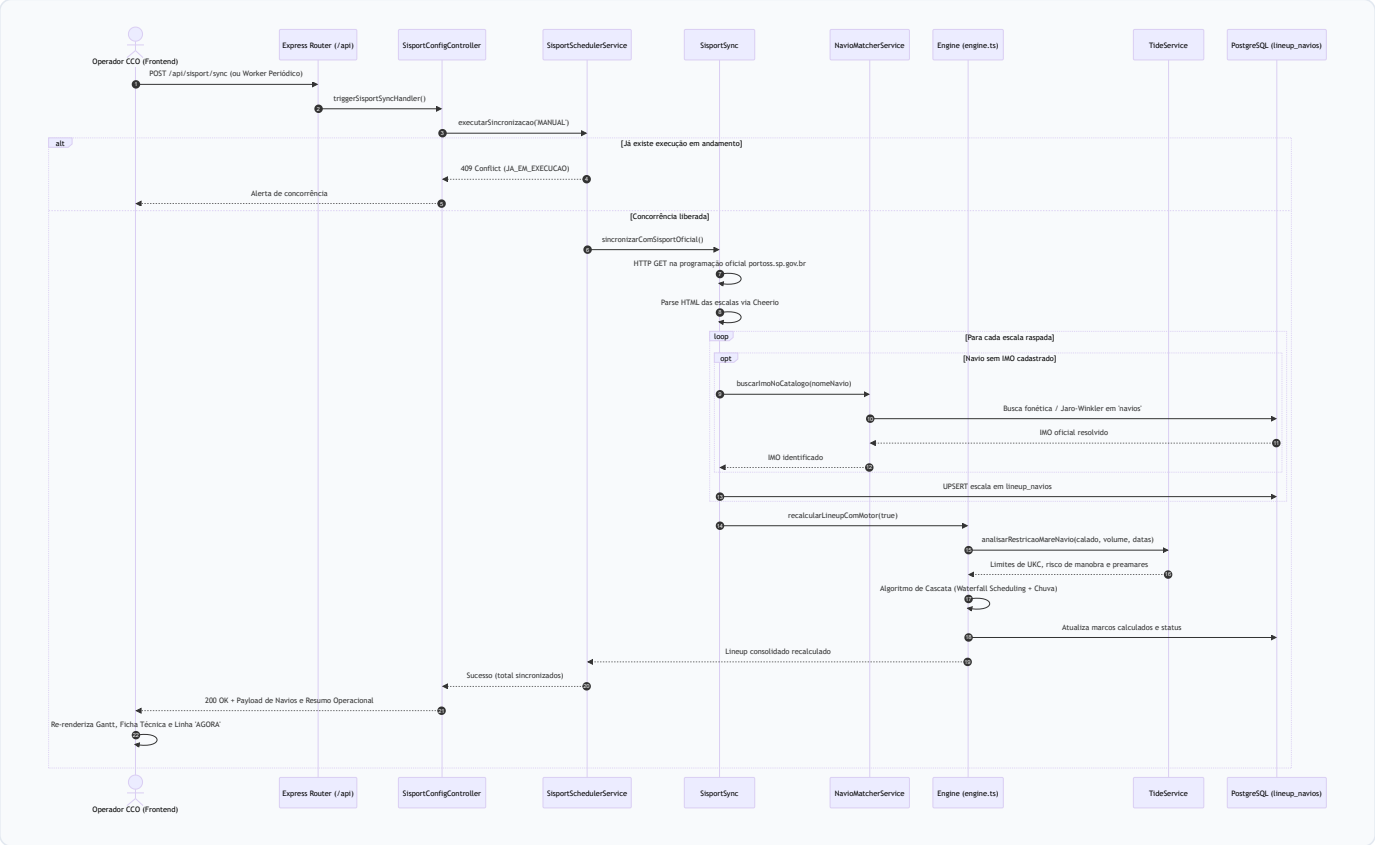
5. Diagrama de Arquitetura do Sistema

O diagrama a seguir mapeia todos os componentes ativos da solução, detalhando o tráfego de dados e integrações:



6. Diagrama de Sequência — Fluxo Operacional Crítico

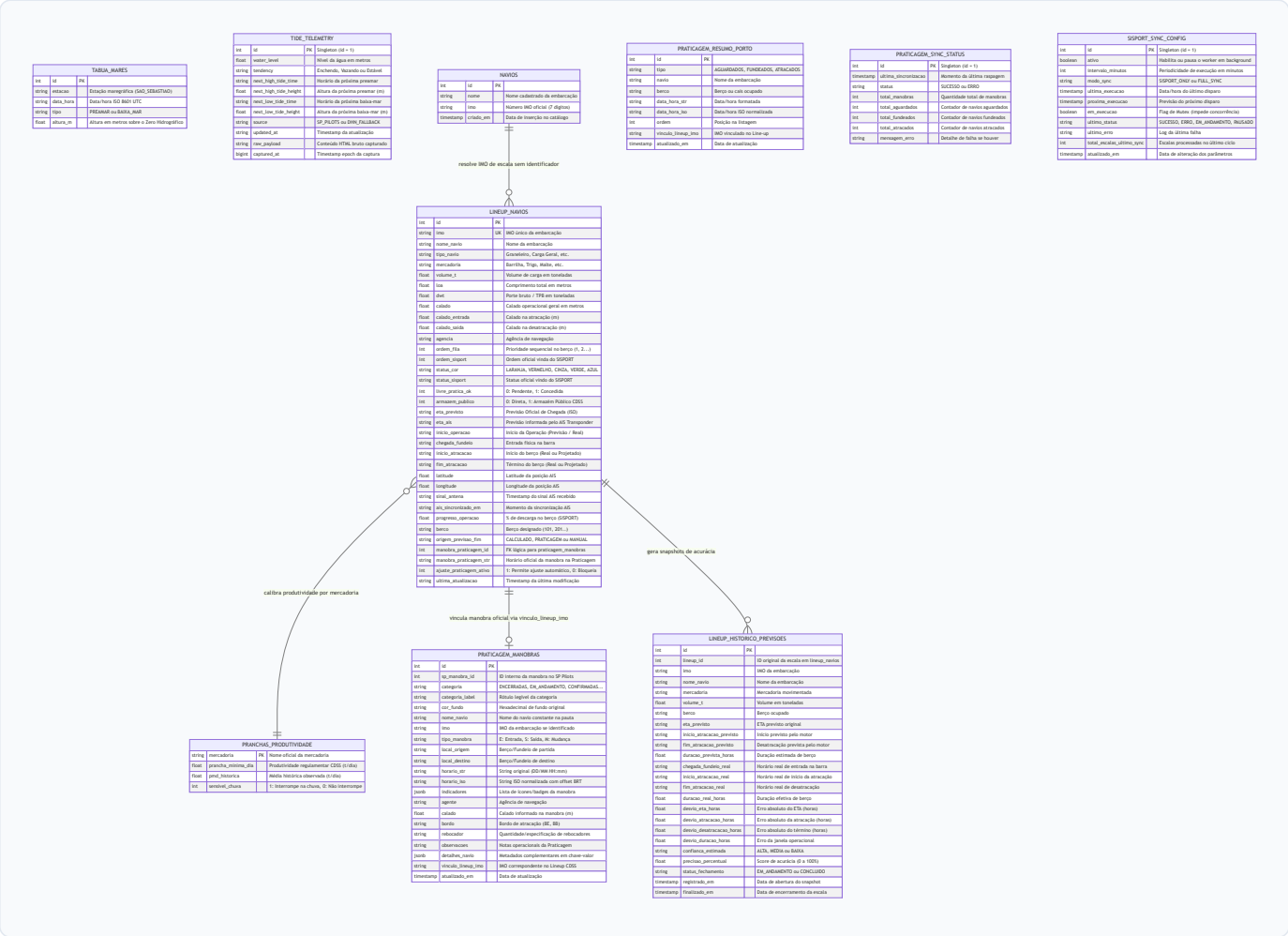
O fluxo a seguir ilustra o ciclo de uma sincronização com o SISPORT, resolução fonética de IMO, enriquecimento por telemetria, encadeamento em cascata no motor determinístico e atualização da interface:



7. Modelo de Dados (PostgreSQL)

O banco de dados é inicializado automaticamente pelo script `src/database/init.ts` na subida do backend, garantindo que todas as tabelas, índices e migrações incrementais estejam aplicadas.

7.1. Diagrama ERD (Entity-Relationship)



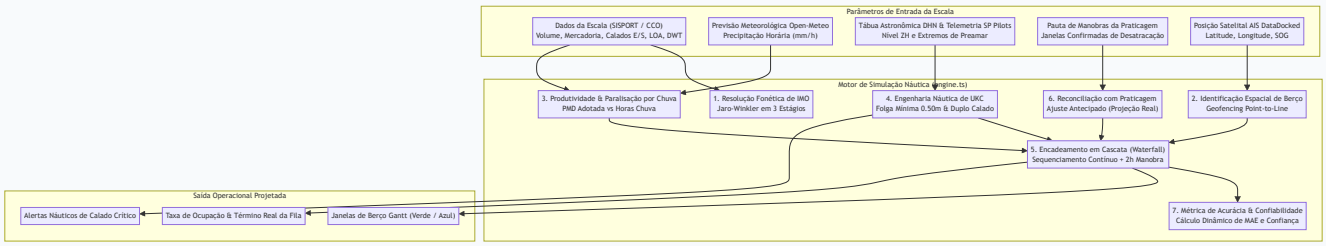
7.2. Descrição das Entidades e Regras de Integridade

1. **`lineup_navios`:** Tabela mestra do line-up do porto. A coluna `imo` possui restrição `UNIQUE`. O status operacional segue a máquina de estados:
   - `LARANJA` (*Previsto em Trânsito*): Navio ainda em aproximação, sem entrada na barra.
   - `VERMELHO` (*Fundeado Aguardando*): Navio com `chegada_fundeio` informada, sem livre prática.
   - `CINZA` (*Pronto para Atracar*): `livre_pratica_ok = 1`, aguardando desocupação de berço.
   - `VERDE` (*Em Operação Real*): Navio atracado fisicamente no berço.
   - `AZUL` (*Previsão de Atracação*): Janela projetada de berço pelo motor determinístico.
2. **`pranchas_produtividade`:** Cadastro regulamentar CDSS. Chave primária em `mercadoria`. Permite diferenciar cargas que paralisam com chuva (`sensivel_chuva = 1`) e estabelece a produtividade mínima diária regulamentar e a média histórica observada.
3. **`tabua_mares`:** Armazena os picos de maré astronômica oficial da Marinha do Brasil (DHN) com chave composta única `(estacao, data_hora)`.
4. **`tide_telemetry`:** Tabela *singleton* (`id = 1`) que preserva a última leitura de nível d'água capturada dos sensores da Praticagem de SP.
5. **`navios`:** Catálogo de 16.200+ embarcações construído a partir de `docs/Navios.csv`. Possui índices `idx_navios_nome` (com `UPPER(nome)`) e `idx_navios_imo` para buscas ultrarrápidas (<1ms) durante a raspagem.
6. **`praticagem_manobras`:** Espelho da pauta oficial do SP Pilots com índices por `categoria`, `nome_navio` e `imo`.
7. **`sisport_sync_config`:** Tabela *singleton* (`id = 1`) que mantém o estado reconfigurável do worker automático em background, registrando `em_execucao` para evitar *race conditions*.
8. **`lineup_historico_previsoes`:** Repositório analítico de histórico de escalas para cálculo de MAE (*Mean Absolute Error*) e confiabilidade estatística do motor de agendamento.

8. Motor Algorítmico Preditivo & Inteligência Náutica

> ℹ️ **Nota de Engenharia:** Esta aplicação **não utiliza LLMs ou modelos de inteligência artificial generativa baseados em prompts** (como OpenAI GPT, Claude ou Gemini). No lugar disso, ela implementa um **Motor Algorítmico Preditivo e de Simulação Marítima Determinístico** (`src/services/engine.ts`), combinando cálculos de engenharia de tráfego aquaviário, resolução fonética de entidades e geofencing espacial.

8.1. Módulos de Inteligência Determinística



8.2. Fórmulas e Regras Matemáticas Implementadas

1. Duração Base de Operação e Prancha Mínima

A produtividade horária é determinada pelo maior valor entre a prancha histórica observada e a mínima regulamentar CDSS: $$\text{PMD}*{\text{adotada}} = \max\left(\text{PMD}*{\text{histórica}}, \text{Prancha Mínima CDSS}\right) \quad [\text{t/dia}]$$ $$\text{Ritmo Horário Base } (R_h) = \frac{\text{PMD}*{\text{adotada}}}{24} \quad [\text{t/h}]$$ $$\text{Duração Base } (D*{\text{base}}) = \frac{\text{Volume Total } (V)}{R_h} \quad [\text{horas}]$$

2. Fator de Paralisação por Chuva (Granéis Sensíveis)

Para mercadorias com `sensivel_chuva = 1` (ex.: Barrilha, Trigo, Malte, Cevada, Fertilizantes), o motor consulta a previsão horária do Open-Meteo. Cada hora com precipitação $> 0.2,\text{mm/h}$ dentro da janela operacional acrescenta exatamente $1.0,\text{hora}$ de interrupção: $$D_{\text{operacional}} = D_{\text{base}} + \Delta T_{\text{chuva}} \quad [\text{horas}]$$

3. Progresso em Berço, Ritmo Efetivo e Tempo Restante (Run-Rate)

Para navios em operação ativa (`VERDE`), o motor analisa o percentual de descarga reportado pelo SISPORT ($P_{%}$): $$\text{Volume Restante } (V_{\text{restante}}) = V \times \left(1 - \frac{P_{%}}{100}\right) \quad [\text{toneladas}]$$ $$\text{Tempo Restante } (T_{\text{restante}}) = \frac{V_{\text{restante}}}{R_h} + \Delta T_{\text{chuva residual}} \quad [\text{horas}]$$

4. Engenharia Náutica de Marés e Folga Abaixo da Quilha (UKC)

O calado máximo admissível no Cais Comercial (Berço 101) depende da cota sobre o Zero Hidrográfico ($\text{ZH} = 10.0,\text{m}$) e da maré instantânea $h_{\text{maré}}(t)$: $$\text{Profundidade Total } (P_{\text{total}}) = 10{,}0,\text{m} + h_{\text{maré}}(t)$$ $$\text{UKC}(t) = P_{\text{total}} - \text{Calado}_{\text{operacional}}$$

- Se $\text{UKC} \ge 0{,}50,\text{m}$: Operação **Segura** (Verde).
- Se $0{,}30,\text{m} \le \text{UKC} < 0{,}50,\text{m}$: **Atenção Náutica** (Amarelo).
- Se $\text{UKC} < 0{,}30,\text{m}$: **Calado Crítico / Restrição** (Vermelho). O sistema avalia de forma independente o **Calado de Entrada** (na atracação) e o **Calado de Saída** (na desatracação), indicando janelas ideais de preamar quando o calado ultrapassa o limiar de segurança de $8{,}5,\text{m}$.

5. Encadeamento em Cascata (Waterfall Scheduling)

O berço comercial 101 aceita apenas uma embarcação comercial por vez. Entre a desatracação de um navio e a atracação do próximo na fila, o motor aplica a constante regulamentar de manobra: $$\Delta T_{\text{manobra}} = 2{,}0,\text{horas}$$ $$\text{Início Atracação}*{n} = \max\left(\text{ETA}*{n}, \text{Fim Atracação}_{n-1} + 2{,}0,\text{h}\right)$$

6. Reconciliação com a Praticagem (Projeção Real vs. Calculada)

Se a Praticagem de SP publicar uma manobra oficial de Saída (`S`) ou Entrada (`E`) confirmada para o navio, e o campo `ajuste_praticagem_ativo = 1`, o motor substitui a projeção matemática calculada pelo horário exato confirmado pelos práticos, propagando o impacto em cascata para todos os navios subsequentes da fila.

---

9. Guia de Inicialização & Ambiente Local

9.1. Pré-requisitos

- **Node.js:** Versão 20.x ou 22.x LTS instalada.
- **npm:** Versão 10.x ou superior.
- **PostgreSQL:** Instância local ou contêiner Docker rodando na porta 5432.
- **Docker & Docker Compose** (opcional, para execução totalmente conteinerizada).

9.2. Instalação e Execução Passo a Passo

```
# 1. Clonar o repositório
git clone https://github.com/alissonvs/CDSS-Lineup.git
cd CDSS-Lineup

# 2. Instalar dependências da raiz e do cliente React (acionado via postinstall)
npm install

# 3. Criar e configurar o arquivo de ambiente
cp .env.example .env
# Edite o .env configurando DATABASE_URL ou as variáveis POSTGRES_*

# 4. Iniciar o banco de dados via Docker Compose (caso não possua PostgreSQL local)
docker compose up -d postgres

# 5. Executar a suíte de testes unitários para validar a integridade matemática
npm run test

# 6. Iniciar o ambiente de desenvolvimento integrado (Backend + Frontend em paralelo)
npm run dev
```

O comando `npm run dev` utiliza `concurrently` para subir simultaneamente:

- **Backend:** `tsx watch src/server.ts` escutando em `http://localhost:3000`.
- **Frontend:** Vite dev server rodando em `http://localhost:5173`.

> 💡 **Nota de Inicialização do Banco:** O script `initializeDatabase()` roda automaticamente na primeira inicialização, criando todas as tabelas, aplicando migrações defensivas e populando o catálogo com 16.200+ embarcações a partir de `docs/Navios.csv` caso a tabela `navios` esteja vazia.

9.3. Scripts Úteis do `package.json`

Script	Comando	Finalidade
`npm run dev`	`concurrently ...`	Executa servidor Express e Vite dev server em paralelo com hot reload.
`npm run dev:server`	`tsx watch src/server.ts`	Executa apenas o backend com reinicialização automática em alterações.
`npm run dev:client`	`npm --prefix client run dev`	Executa apenas o frontend Vite em modo desenvolvimento.
`npm run test`	`vitest run`	Executa todos os 39 testes unitários determinísticos de motor e parsers.
`npm run build`	`npm run build:server && npm run build:client`	Compila o backend TypeScript (`dist/`) e o frontend Vite (`client/dist/`).
`npm run start`	`node dist/server.js`	Executa o servidor de produção compilado.
`npm run sync:sisport`	`tsx src/scripts/syncSisport.ts`	Dispara manualmente o scraper do SISPORT via linha de comando.
`npm run import:navios`	`tsx src/scripts/importNavios.ts`	Força a importação do catálogo `docs/Navios.csv` para a tabela `navios` .

---

10. Deploy em Produção (Docker & Dokploy)

A aplicação foi projetada para deploy simplificado em provedores modernos como **Dokploy**, **CapRover**, **Railway** ou instâncias **VPS Docker**.

10.1. Build Multi-Stage com Dockerfile

O Dockerfile utiliza arquitetura multi-stage em dois estágios:

1. **Estágio `builder` (`node:22-alpine`):** Instala dependências de desenvolvimento, compila o servidor TypeScript para `dist/` e executa o bundle do Vite para `client/dist/` .
2. **Estágio `runner` (`node:22-alpine`):** Imagem final enxuta contendo apenas dependências de produção (`npm ci --omit=dev`), rodando com o usuário não-root `node` para segurança máxima.

10.2. Subida com Docker Compose

Para subir toda a infraestrutura em produção local ou VPS com um único comando:

```
docker compose up -d --build
```

O contêiner da aplicação aguarda a saúde do banco de dados através da instrução `depends_on` vinculada ao healthcheck `pg_isready` :

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-cdsslineup} -d ${POSTGRES_DB:-cdsslineup}"]
  interval: 5s
  timeout: 5s
  retries: 5
```

10.3. Configuração de Deploy no Dokploy

1. **Repositório:** Conecte o repositório Git no painel do Dokploy.
2. **Build Type:** Selecione `Dockerfile` .
3. **Porta do Serviço:** Configure para a porta interna `3000` .
4. **Variáveis de Ambiente:** Cole o bloco de variáveis no Dokploy:

```
NODE_ENV=production
PORT=3000
HOST=0.0.0.0
DATABASE_URL=postgresql://usuario:senha@host-do-banco:5432/nome_do_banco
SISPORT_AUTO_SYNC_ENABLED=true
SISPORT_SYNC_INTERVAL_MINUTES=15
SISPORT_SYNC_MODE=SISPORT_ONLY
VITE_CARTO_API_KEY=sua_chave_carto
DATADOCKED_API_KEY=sua_chave_datadocked
```

5. **Healthcheck Probe:** O contêiner expõe a rota nativa de diagnóstico:
   - Endpoint: `http://127.0.0.1:3000/api/health`
   - Retorno esperado: HTTP 200 com status `healthy`.

---

11. Pontos de Atenção & Inconsistências Identificadas

Durante a análise técnica aprofundada da base de código, foram identificados os seguintes itens que merecem atenção por parte da equipe de engenharia:

1. **Variáveis de Ambiente Alternativas vs. `.env.example`:**
   - O arquivo `src/database/connection.ts` aceita `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` e `DB_NAME`, porém o `.env.example` lista apenas as variantes `POSTGRES_*` e `DATABASE_URL`. Recomenda-se unificar as orientações.
2. **Dependência Externa de Chave CARTO em Build Time:**
   - O `Dockerfile` possui o argumento `ARG VITE_CARTO_API_KEY`. Como o Vite embute variáveis prefixadas com `VITE_` em tempo de compilação estática (`client/dist`), alterar a chave do CARTO exige um novo build do contêiner, não sendo suficiente apenas alterar a variável de ambiente do contêiner em runtime.
3. **Tratamento de Rate-Limiting em Raspagem HTML:**
   - As integrações com o SISPORT e Praticagem realizam requisições periódicas a servidores governamentais e de serviços marítimos. Recomenda-se manter o intervalo mínimo configurado em $\ge 15,\text{minutos}$ para evitar bloqueios temporários de IP (*WAF/firewall* das entidades).
4. **Reserva de Memória no Parsing do `Navios.csv`:**
   - O arquivo `docs/Navios.csv` contém mais de 16.200 linhas (350+ KB). A carga inicial lê e insere esses registros no banco. Em ambientes com memória muito restrita (<512MB RAM), a carga em lote deve ser monitorada.
5. **Conexões HTTP sem Autenticação de Borda:**
   - A API REST atualmente não exige tokens JWT ou cabeçalhos de autorização em rotas de mutação (`POST /api/navios`, `PUT /api/navios/:id`, `DELETE /api/navios/:id`), operando sob a premissa de rede local fechada / VPN do CCO. Para exposição pública na internet, recomenda-se adicionar um middleware de autenticação ou proteger a aplicação atrás de um Reverse Proxy com autenticação básica ou OAuth2/SSO.